

Idea-as-a-Service: Low-Budget POC Plan

How to prove the core loop (idea to working, tested, deployed app) for \$0 to about \$900 instead of \$300k+, and what to cut until it is proven.

Assumption. "IaaS" here means **Idea-as-a-Service** from your blueprint. It does not mean Infrastructure-as-a-Service. If you meant the latter, this plan needs to be redone.

1. What the POC must prove (and nothing else)

The full blueprint is a factory for ERP, web, mobile, IoT, network and cyber products. A POC should not build a factory. It should answer **one question** with evidence:

Can an AI pipeline, with a human approving three checkpoints, turn a one-page idea into a working, tested web app on a staging URL, at a cost and effort a small team could afford?

The narrow loop:

```
Idea (1 page) -> PRD + typed contracts (schema-validated) -> tests first -> code in sandbox
-> verify (syntax, tests, lint, security scan) -> auto-repair (max 3 loops) -> staging URL
Human gates: (1) approve PRD (2) approve contracts (3) approve staging build
```

In scope

- One stack (recommended: FastAPI or Node + Postgres/SQLite + a simple React or server-rendered UI) and one demo domain (for example equipment rental or a small-business CRM).
- A schema gate: every AI output that later steps depend on is validated against a typed schema before use.
- A sandboxed build-and-test step and a bounded repair loop.
- Cost and token tracking per project.

Explicitly out of scope

- Mobile, IoT, network and cyber squads; compliance logic; multi-tenant billing; custom domains; maintenance automation; Kubernetes, Temporal, Firecracker, dedicated GPUs.

2. Where the blueprint is cut down

Blueprint layer	POC substitute	Cost	Revisit when
8-GPU inference node	Hosted API for open-weight coder models behind one router function (OpenRouter / DeepSeek / Gemini Flash), or local Ollama (qwen2.5-coder 7B-14B) for \$0. A frontier model only for planning and escalations.	\$0 to a few \$ per project	API spend passes about \$500/mo, or privacy demands local
Temporal + Kubernetes control plane	Plain Python or Node state machine, state persisted in SQLite/Postgres (Docker Compose)	\$0	Runs must survive restarts or run in parallel
Firecracker / gVisor sandbox	Rootless Docker, no network by default, CPU/memory/time limits (or a child process with a timeout for the first spike)	\$0	You run untrusted third-party code

Blueprint layer	POC substitute	Cost	Revisit when
Type-safe decoding (grammar-constrained)	JSON Schema / Zod / Pydantic validation plus re-ask on failure; provider JSON mode where available	\$0	Retry rate is high enough to cost real money
Full SAST/DAST suites	Language syntax check, unit/integration tests, Semgrep + Gitleaks + Trivy (all free)	\$0	Before any real customer data
GitLab + CI farm + HA observability	Forgejo or GitHub free; Langfuse as a single container, or a CSV log of tokens and cost	\$0	More than about 5 users
Multi-region staging	Local live preview; then one small VPS or a free tier (Cloudflare, Vercel, Fly.io)	\$0 to \$10/mo	Real users
5-FTE platform team	You, plus optionally one part-time helper. Your time is the real cost.	Your time	POC passes the gate

3. Design choices that keep it cheap

- **Template first, code second.** Start with one hand-built, known-good app template (auth, CRUD, dashboard). Let the AI produce a *validated specification and configuration* plus targeted code for the parts that differ. This cuts tokens, failure rate and review effort sharply. Move to freer code generation only after the template path works.
- **One deployed app can host many configuration-driven "generated" apps** (routes like `/apps/<slug>`). This avoids per-app infrastructure in the demo. It is a demo shortcut, not the final architecture.
- **Tests before code.** Ask for tests from the contracts first. A failing test is a cheap, objective signal for the repair loop and for measuring quality.
- **Bounded loops.** Max 3 repair attempts and a token budget per stage. When exceeded, stop and hand over to a human. Uncapped agents are the main way a low-budget POC turns into a high bill.
- **Model router.** Every LLM call goes through one function that takes a role (`plan`, `code`, `repair`, `review`). Swap models per role without touching the pipeline.

4. Budget options (8 weeks, including about 20% contingency)

Option	Total	What you get	Best for
Tier 0: Local-only	about \$0 to \$30	Ollama on your existing machine (7B-14B coder model), SQLite, local Docker, local preview. Slower and lower quality, but proves the pipeline mechanics.	Weeks 1-2 spike, offline demos
Tier 1: Lean (recommended start)	about \$220	Hosted open-weight model APIs (about \$80-120), a small VPS for staging and Forgejo (about \$10-20/mo), optional domain, contingency. No frontier API, no GPU rental.	Weeks 1-6
Tier 2: Standard	about \$875	Lean plus frontier-model planning and escalation (about \$300-400) and about \$150 of rented-GPU benchmarking to test self-hosting economics.	Weeks 7-8, only if the gate passes
Tier 3: Stretch	about \$2,500	Standard plus a 3-day rental of an 8-GPU node to test the blueprint's target serving stack under real load.	Only for investor or partner evidence

All prices and token costs are planning estimates for 2026, not quotes. Week 3 measures real cost per project; use those numbers, not these, for any commitment. Compare with the blueprint's Tier A pilot capex of \$20-45k plus operating costs: the POC defers all of it until the loop is proven. An itemised breakdown is in `POC_Budget.csv`.

5. Eight-week build plan

Week	Goal	Deliverable / exit check
1	Pick stack and demo domain; write 3 test ideas of increasing difficulty; set up repo, Docker Compose, and a model router with 2 providers.	"Hello pipeline": a stub run goes through every stage and logs cost.
2	Idea to PRD to typed contracts (entities, endpoints, acceptance criteria) with schema validation and re-ask.	3 ideas produce valid contracts; invalid output is caught every time.
3	Tests-first code generation on the template; sandbox runner. Measure real token cost.	One idea yields passing tests locally; cost per run recorded.
4	Repair loop (max 3), static checks (Semgrep, Gitleaks), failure taxonomy.	Repair loop fixes a majority of first-run failures.
5	Human gates (approve PRD, contracts, build), staging deploy, minimal web UI or CLI for the whole flow.	End-to-end run to a public or LAN staging URL.
6	Run all 3 ideas 3 times each; collect metrics. Go / no-go gate.	Metrics table (section 6) and a written decision.
7-8	If go: add a second stack or domain, frontier planning, GPU cost benchmark, demo video, pitch. If no-go: write up why and what to change.	Demo, cost model, next-phase plan.

6. Week-6 gate: pass / kill thresholds

Metric	Pass	Kill or rethink
Ideas reaching a working staging app (of 3)	2 or more	0 or 1
Generated tests that pass (after repair)	85% or more	below 60%
Extra human interventions per project (beyond the 3 gates)	fewer than 5	10 or more
Cost per project (tokens plus compute)	under \$100	over \$250
Time from idea to staging	under 4 hours	more than 2 days
Schema-invalid output reaching later stages	0	any

Between pass and kill: keep going two more weeks on Lean only, fix the top failure category, and re-measure.

7. Risks

Risk	Mitigation
Scope creep back to the full blueprint	The out-of-scope list in section 1 is binding until the gate passes.
Small local models produce poor code	Template-first, tests-first, hosted open-weight or frontier fallback for hard steps.
Runaway token spend	Per-stage budgets, max repair loops, a hard monthly cap and daily spend alert on the API key.
Running generated code unsafely	Rootless Docker, no network, resource limits, never mount host secrets.
Demo works only on the happy path	3 ideas of different difficulty, 3 runs each, report failures honestly.
"It works" without a business case	Interview 5 target users before week 5; record what they would pay for.

Risk	Mitigation
Price assumptions are stale	Re-check provider pricing at week 1 and week 3.

8. This week's to-do list

1. Choose the stack and the demo domain; write the three test ideas (easy, medium, hard).
2. Install Docker and Ollama with one coder model; create one hosted-model API account with a hard monthly spend cap.
3. Create the repo and Compose file; write the model router with two providers.
4. Write the "project spec" schema (entities, fields, endpoints, acceptance criteria) and a validator.
5. Build one hand-made template app and its test suite; this is the target the AI fills in.
6. Get one stub run through all stages, logging tokens and cost.

9. After the POC

If the gate passes, follow the blueprint's roadmap in order: Tier A pilot infrastructure, Temporal for durable runs, Firecracker/gVisor for isolation, a second squad (for example ERP), and only then mobile, IoT, network and cyber. If it fails, the failure data tells you whether the problem is model quality, spec quality or verification, and each has a different fix.

Companion files: `POC_Budget.csv` (itemised costs by tier), `Idea-as-a-Service-Plan.pdf` (full platform blueprint).